

Systemy czasu rzeczywistego, Projekt 4 — Tło teoretyczne

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

Czerwiec 2026

Dodatek A — Tło teoretyczne

A.0 Słowniczek — co znaczą podstawowe pojęcia

Magistrala (*bus*) — wspólna para przewodów łącząca wszystkie urządzenia w sieci. Każdy sygnał jest widoczny przez wszystkich — jak tablica ogłoszeń, z której każdy może czytać.

Węzeł (*node*) — jedno urządzenie podłączone do magistrali (czujnik, sterownik, komputer).

Ramka (*frame*) — jedna paczka danych. Poza właściwymi danymi zawiera nagłówek (numer wiadomości, ile jest danych), sumę kontrolną i inne pola. Jak koperta pocztowa: prócz listu jest adres, znaczek itp.

Stany recessive i dominant. Magistrala CAN ma w każdej chwili jeden z dwóch stanów:

- *Recessive* (z łac. cofający się) = logiczne 1 — stan *bierny*: żadne urządzenie nic nie robi, magistrala „odpoczywa” na napięciu $\approx 2,5\text{ V}$ na obu przewodach.
- *Dominant* (z łac. panujący) = logiczne 0 — stan *aktywny*: co najmniej jedno urządzenie aktywnie steruje napięciami na przewodach.

Zasada nadrzędna: jeden dominant bierze górę nad wszystkimi recessive. Jeśli urządzenie A wysyła dominant (0) jednocześnie gdy B wysyła recessive (1), na magistrali widać 0. Jak głośny krzyk zagłuszający szept.

Sygnalowanie różnicowe — skąd nazwa? Zwykły kabel cyfrowy przesyła napięcie jednego przewodu względem masy: 0 V to zero, 3,3 V to jedynek. Problem: każdy impuls elektromagnetyczny (od silnika, telefonu, błyskawicy) bezpośrednio zmienia to napięcie.

CAN używa *dwóch* przewodów: CAN_H (High) i CAN_L (Low). Informacja jest zakodowana w **różnicy** napięć między nimi:

Stan	CAN_H	CAN_L	Różnica H–L
<i>recessive</i> (1)	$\approx 2,5\text{ V}$	$\approx 2,5\text{ V}$	$\approx 0\text{ V}$
<i>dominant</i> (0)	$\approx 3,5\text{ V}$	$\approx 1,5\text{ V}$	$\approx 2\text{ V}$

Zakłócenie uderza jednakowo w oba przewody biegnące obok siebie — podnosi napięcie na obu o tę samą wartość, więc różnica H–L pozostaje niezmienną. Stąd nazwa: **różnicowe** — liczymy różnicę, nie wartość bezwzględną.

Ciągnięcie napięcia. W stanie recessive nikt nic nie robi: rezystory zamontowane na końcach kabla (rezystory terminacyjne) utrzymują oba przewody pasywnie na $\approx 2,5\text{ V}$. W stanie dominant układ scalony (transceiver — nadajnik/odbiornik CAN) włącza wewnętrzne tranzystory, które:

- **ciągną CAN_H w górę** — ku napięciu zasilania ($\approx 3,5\text{ V}$),
- **ciągną CAN_L w dół** — ku masie ($\approx 1,5\text{ V}$).

„Ciągnięcie” to standardowy termin elektroniczny: tranzystor musi dostarczyć prąd, żeby wymusić napięcie wbrew rezystorom — jak ciągnięcie liny wbrew sprężynie.

Arbitracja — kto nadaje pierwszy? CAN to sieć bez żadnego nadrzędnego koordynatora: każde urządzenie może zacząć nadawać w dowolnej chwili. Gdy dwa węzły zaczną jednocześnie — kolizja.

CAN rozwiązuje ją automatycznie: każde nadające urządzenie *śłucha* magistrali podczas własnej transmisji i porównuje to co wysyła z tym co widzi. Numer ID wiadomości jest wysyłany bit po bicie od najstarszego. Jeśli urządzenie wysłało 1 (recessive), ale na magistrali widzi 0 (dominant) — znaczy to, że inne urządzenie wysyła tam 0, czyli ma niższy numer ID. Urządzenie z wyższym ID milknie i czeka. Wygrywa ten z niższym numerem ID — i nie traci przy tym żadnych danych, bo dominanty wygrywają bez zniszczenia ramki. Stąd: **niższy numer ID = wyższy priorytet**.

Bit stuffing (wstawianie bitów synchronizujących). Odbiornik nie ma osobnego kabla zegarowego. Synchronizuje swój wewnętrzny zegar obserwując *zmiany* stanu na magistrali — każde przejście $0 \rightarrow 1$ lub $1 \rightarrow 0$ jest jak uderzenie metronomu. Problem: przy długim ciągu identycznych bitów (np. 10 jedynek z rzędu) przez cały ten czas nie ma żadnej zmiany i odbiornik może „zgubić rytm”.

Rozwiązanie: nadajnik automatycznie **wstawia jeden odwrócony bit po każdych 5 bitach tej samej wartości**. Odbiornik zna tę zasadę i usuwa te dodatkowe bity. Takie bity pomocnicze to **bity stuffujące**.

CRC (Cyclic Redundancy Check — cykliczna suma kontrolna). Matematyczna metoda wykrywania przekłamań. Nadajnik oblicza 15-bitową liczbę kontrolną ze wszystkich bitów wiadomości i dołącza ją na końcu. Odbiornik oblicza tę samą liczbę ze *odebranych* bitów i porównuje — jeśli różne, gdzieś nastąpił błąd. Szczegóły w A.5.

Skróty pól ramki: SOF, EOF, IFS, ACK, DLC, SRR, IDE, RTR — każde wyjaśnione przy tabeli w A.2.

A.1 Sieć CAN — podstawy

CAN (*Controller Area Network*) to standard komunikacyjny opracowany przez firmę Bosch w 1986 roku, pierwotnie na potrzeby motoryzacji. Dziś stosowany jest również w automatyce przemysłowej, medycynie i robotyce.

Kluczowe właściwości:

- **Każdy może nadawać w dowolnej chwili** — nie ma żadnego centralnego koordynatora. Konflikty przy jednoczesnym nadawaniu rozstrzyga arbitracja (patrz A.0): wygrywa wiadomość z niższym numerem ID, bez straty danych.
- **Odporność na zakłócenia elektromagnetyczne** — dzięki sygnałowaniu różnicowemu (dwa przewody CAN_H i CAN_L; szczegóły w A.0).
- **Dwa stany magistrali:** *dominant* (logiczne 0, aktywnie sterowane napięcia) i *recessive* (logiczne 1, stan bierny). Jeden dominant zawsze wygrywa nad recessive (szczegóły w A.0).
- **Prędkość:** do 1 Mb/s (klasyczny CAN), do 8 Mb/s (CAN FD).

Priorytety: numer ID wiadomości decyduje o priorytecie. Niższy numer ID = więcej zer (dominantów) na początku identyfikatora = wygrywa arbitrację. Wiadomości krytyczne (np. czujnik poduszki powietrznej) dostają niskie numery ID i zawsze trafiają na magistralę jako pierwsze.

A.2 Struktura rozszerzonej ramki CAN (CAN 2.0B)

CAN 2.0 definiuje dwa formaty ramek — różnią się długością numeru ID:

- **Standardowy** (CAN 2.0A) — 11-bitowy numer ID (do 2032 różnych typów wiadomości),
- **Rozszerzony** (CAN 2.0B) — 29-bitowy numer ID (ponad 532 miliony typów wiadomości).

Zadanie 4.3 dotyczy ramki rozszerzonej z 8 bajtami danych. Każde **pole** to wydzielony fragment ramki o ustalonej liczbie bitów i ściśle określonej roli:

Pole	Podpole	Bitów	Co to jest
SOF		1	<i>Start of Frame</i> — jeden bit dominant (0) otwierający ramkę. Magistrala jest normalnie recessive (1); nagle 0 budzi wszystkich odbiorców: „zaraz przyjdzie wiadomość”.
Arbitration	Base ID [28:18]	11	Starsze 11 bitów 29-bitowego numeru ID wiadomości.
	SRR	1	<i>Substitute Remote Request</i> — zawsze recessive; techniczny bit zgodności ze starszym formatem 11-bitowym, bez znaczenia merytorycznego.
	IDE	1	<i>Identifier Extension</i> — wartość recessive (1) mówi odbiornikowi: „to jest ramka rozszerzona (29-bitowy ID), nie standardowa (11-bitowy ID)”.
	Extension ID [17:0]	18	Młodsze 18 bitów 29-bitowego numeru ID.
	RTR	1	<i>Remote Transmission Request</i> — gdy ustawiony na 1, ramka jest <i>prośbą</i> o dane od innego węzła, nie ich wysłaniem. Tu zawsze 0 (wysyłamy dane, nie prosimy).
Control	r1	1	Bit zarezerwowany dla przyszłych wersji standardu; zawsze 0.
	r0	1	Drugi bit zarezerwowany; zawsze 0.
	DLC [3:0]	4	<i>Data Length Code</i> — liczba (0–8) mówiąca, ile bajtów właściwych danych zawiera pole Data.
Data		64	Właściwe dane: 8 bajtów $\times$ 8 bitów = 64 bity.
CRC	CRC sequence	15	15-bitowa suma kontrolna obliczona z wszystkich wcześniejszych pól. Odbiornik liczy tę samą sumę i porównuje — niezgodność oznacza błąd transmisji.
	CRC delimiter	1	Jeden bit recessive (1) oddzielający sumę kontrolną od potwierdzenia odbioru — jak spacja między słowami.
ACK	ACK slot	1	Potwierdzenie odbioru: nadajnik wysłała tu recessive (1), ale <i>każdy odbiorca, który poprawnie odebrał ramkę</i> , nadpisuje to dominant (0). Jeśli nadajnik widzi na magistrali 0 — wie, że ktoś odebrał wiadomość poprawnie.
	ACK delimiter	1	Jeden bit recessive (1) kończący pole ACK.
EOF		7	<i>End of Frame</i> — 7 bitów recessive (samych jedynek) oznaczających koniec ramki.
IFS		3	<i>Interframe Space</i> — obowiązkowe 3 bity przerwy między ramkami, dające wszystkim węzłom czas na przetworzenie właśnie odebranej wiadomości za-

Łączna liczba bitów dla $DLC = 8$ (8 bajtów danych):

$$L_{\text{stały}} = \underbrace{1}_{\text{SOF}} + \underbrace{(11 + 1 + 1 + 18 + 1)}_{\text{Arbitration}=32} + \underbrace{(1 + 1 + 4)}_{\text{Control}=6} + \underbrace{64}_{\text{Data}} + \underbrace{(15 + 1)}_{\text{CRC}=16} + \underbrace{(1 + 1)}_{\text{ACK}=2} + \underbrace{7}_{\text{EOF}} + \underbrace{3}_{\text{IFS}}$$

$$= 1 + 32 + 6 + 64 + 16 + 2 + 7 + 3 = \mathbf{131} \text{ bitów}$$

Do tej liczby dochodzą jeszcze **bity stuffujące** (patrz A.3). Bit stuffing stosuje się do regionu: SOF + Arbitration + Control + Data + sekwencja CRC (bez separatora CRC i dalszych pól):

$$N_{\text{region stuffingu}} = 1 + 32 + 6 + 64 + 15 = \mathbf{118} \text{ bitów} \quad (1)$$

A.3 Technika bit stuffing — zasada i zastosowanie

Dlaczego w ogóle potrzebny jest bit stuffing?

Odbiornik nie ma osobnego kabla z sygnałem zegarowym. Synchronizuje swój wewnętrzny zegar obserwując *zmiany* stanu na magistrali — każde przejście $0 \rightarrow 1$ lub $1 \rightarrow 0$ jest jak uderzenie metronomu. Wyobraź sobie odczytywanie Morse’a ze słuchu: jeśli przez długi czas nie słyszysz żadnej zmiany, gubisz rachubę. Jeśli dane zawierają 6 lub więcej identycznych bitów z rzędu (np. sześć jedynek), odbiornik może przesunąć swoje taktowanie i błędnie zinterpretować kolejne bity.

Zasada:

Po każdych **5 bitach tej samej wartości** nadajnik automatycznie wstawia jeden bit o *przeciwnej* wartości. Ten dodatkowy bit nie niesie żadnych danych — służy tylko do podtrzymania synchronizacji. Odbiornik zna tę zasadę i usuwa co szósty bit, jeśli poprzedzały go pięć identycznych. Takie bity pomocnicze to **bity stuffujące**.

Przykład:

Dane do wysłania	1	1	1	1	1	0	0	0	0	0	1	1	
Na magistrali (po wstawieniu)	1	1	1	1	1	[0]	0	0	0	0	[1]	1	1

Dodane bity zaznaczono nawiasami []. Odbiornik je usuwa i odtwarza oryginalne dane.

Gdzie bit stuffing obowiązuje?

- **Tak** (118 bitów łącznie): SOF, pole Arbitration, pole Control, pole Data, sekwencja CRC.
- **Nie**: separator CRC, pole ACK, EOF, IFS — te pola mają z góry ustalone stałe wartości, więc odbiornik nie musi liczyć przejść, bo wie czego się spodziewać.

Ile bitów stuffujących może dojść maksymalnie?

W najgorszym przypadku co 5 bitów wymusza wstawienie jednego stuffującego. Dla regionu 118 bitów:

$$\text{Max. bitów stuffujących} = \left\lfloor \frac{118}{5} \right\rfloor = \mathbf{23} \quad (2)$$

Minimum wynosi 0 — gdy dane są naprzemienne ($10101010\dots$) i nigdzie nie pojawia się 5 identycznych bitów z rzędu.

A.4 Identyfikatory CAN i ograniczenia specyfikacji

W sieci CAN wiadomości nie mają adresata — każda ramka wysłana na magistralę jest widoczna przez wszystkie węzły. Każdy węzeł sam decyduje, czy dana wiadomość go dotyczy, na podstawie **numeru ID**. Węzeł konfiguruje swój kontroler CAN tak, aby reagował tylko na wybrane numery ID (analogia: stacja radiowa — każde radio słyszy wszystkie częstotliwości, ale słychać tylko jedną).

Ile różnych numerów ID istnieje?

- Ramka standardowa (11 bitów na numer ID): $2^{11} = 2048$ kombinacji.
- Ramka rozszerzona (29 bitów na numer ID): $2^{29} = 536\,870\,912$ kombinacji.

Ograniczenie: identyfikatory, których *pierwsze 7 bitów* to wszystkie jedynki (1111111...), są **zarezerwowane i zakazane**. Powód: taka kombinacja wygląda identycznie jak specjalny sygnał błędu (*passive error flag* — 6 recessive) albo jak pole IFS (pauza między ramkami), co mogłoby zmylić odbiorniki.

Format	Bitów ID	Zakazane ID	Dostępne ID
Standardowy	11	$2^4 = 16$	$2048 - 16 = \mathbf{2032}$
Rozszerzony	29	$2^{22} = 4\,194\,304$	$536\,870\,912 - 4\,194\,304 = \mathbf{532\,676\,608}$

Tabela 2: Rzeczywista liczba dostępnych identyfikatorów w sieci CAN.

W praktyce ograniczenie to jest nieistotne: typowa sieć przemysłowa używa kilkudziesięciu do kilkuset różnych typów wiadomości — ułamek procenta dostępnej przestrzeni.

A.5 Suma kontrolna CRC-15/CAN — teoria i gwarancje

Idea: jak CRC wykrywa błędy?

Wyobraź sobie, że wysyłasz liczbę i chcesz sprawdzić, czy dotarła bez zmian. Możesz z góry obliczyć resztę z dzielenia tej liczby przez jakąś ustaloną stałą i dołączyć ją jako „podpis”. Odbiorca dzieli odebraną liczbę przez tę samą stałą — jeśli reszta się zgadza, liczba prawdopodobnie jest nienaruszona. Jeśli nie — gdzieś nastąpiła zmiana.

CRC działa dokładnie tak, ale operuje na bitach i używa specjalnej arytmetyki binarnej (bez przeniesień przy dodawaniu — patrz B.1). Wynikiem jest 15-bitowa reszta z dzielenia — to właśnie suma kontrolna CRC-15.

Formalnie: ciąg bitów wiadomości traktowany jest jak wielomian (każdy bit to współczynnik przy kolejnej potęgze x). Nadajnik liczy resztę z dzielenia wielomianu wiadomości $M(x)$ przez ustalony wielomian generujący $G(x)$:

$$R(x) = M(x) \cdot x^{15} \bmod G(x)$$

i dołącza resztę $R(x)$ do ramki. Odbiornik sprawdza, czy odebrany strumień jest podzielny przez $G(x)$ bez reszty — niezerowa reszta = błąd.

Wielomian generujący CRC-15/CAN

Wielomian generujący to matematyczna stała — rodzaj „dzielnika”. Jego wybór decyduje o tym, jakie błędy CRC jest w stanie wykryć. Dla CAN jest to:

$$G(x) = x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1 \quad (3)$$

W algorytmie z zadania ta stała pojawia się jako liczba szesnastkowa 0x4599 (15 bitów, bez najstarszego bitu x^{15} , który jest niejawny):

$$G_{\text{hex}} = 0x4599 = \underbrace{0100\ 0101\ 1001\ 1001}_{\text{bity } 14 \dots 0}$$

Co CRC-15/CAN gwarantuje?

Wielomian był starannie wybrany przez Boscha pod kątem zakłóceń typowych dla okablowania samochodowego. Gwarantuje wykrycie:

- **każdego błędu 1-bitowego** — jeden przekłamaný bit,
- **każdego błędu 2-bitowego** — dowolne dwa przekłamane bity,
- **każdej serii błędów** o długości do 15 bitów z rzędu,
- **każdej nieparzystej liczby przekłamaných bitów**.

Prawdopodobieństwo, że jakiś błąd *nie* zostanie wykryty, wynosi $\approx 3,1 \times 10^{-5}$ — czyli mniej niż 1 na 32 000 błędnych ramek „prześlizgnie się” niezauważone.

Algorytm LFSR z zadania

Algorytm z treści zadania to sprzętowa implementacja tego dzielenia, zwana LFSR (*Linear Feedback Shift Register* — liniowy rejestr przesuwający ze sprzężeniem zwrotnym). Szczegółowy opis z analogią taśmy produkcyjnej znajduje się w B.2. Zmienne algorytmu:

Zmienna	Co przechowuje
CRC_RG(14:0)	15-bitowy rejestr — bieżąca reszta dzielenia (bit 14 = najstarszy)
NXTBIT	kolejny bit wejściowy (0 lub 1)
CRCNXT	bit pomocniczy = XOR bitu wejściowego z najstarszym bitem rejestru

1. CRC_RG = 0 (inicjalizacja — rejestr wyzerowany)
2. Dla każdego bitu wejściowego NXTBIT (od SOF do końca Data):
 - a) $\text{CRCNXT} = \text{NXTBIT} \oplus \text{CRC_RG}(14)$
 - b) przesun rejestr w lewo o 1: $\text{CRC_RG}(14:1) \leftarrow \text{CRC_RG}(13:0)$; $\text{CRC_RG}(0) = 0$
 - c) jeśli $\text{CRCNXT} = 1$: wykonaj XOR rejestru z 0x4599
3. Wynik: zawartość CRC_RG po przetworzeniu wszystkich bitów.

Dodatek B — Wymagana wiedza i kluczowe aspekty przy implementacji

B.1 Arytmetyka binarna bez przeniesień — dlaczego XOR jest sercem CRC

Normalna arytmetyka: $1 + 1 = 2$ (przeniesienie do następnej cyfry). CRC używa arytmetyki binarnej **bez przeniesień**: $1 + 1 = 0$. To jest dokładnie operacja XOR (exclusive or — „albo jedno, albo drugie, nie oba naraz”):

$$0 \oplus 0 = 0, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0$$

Ta arytmetyka nosi nazwę **GF(2)** (ciało Galois rzędu 2). Kluczowe właściwości: dodawanie i odejmowanie to ta sama operacja (XOR), nie ma przeniesień, nie ma ujemnych liczb. Mnożenie to AND ($1 \cdot 1 = 1$, $1 \cdot 0 = 0$).

Ciąg bitów można interpretować jako wielomian: ciąg 1011 to x^3+x+1 (jedyne na pozycjach 3, 1, 0). Dzielenie takiego wielomianu przez inny w arytmetyce GF(2) daje resztę — to właśnie CRC.

Dwie kluczowe obserwacje dla implementacji:

- Przesunięcie bitów rejestru w lewo o 1 ($\ll 1$) to mnożenie wielomianu przez x .
- XOR rejestru z wielomianem generującym to odjęcie tego wielomianu (w GF(2) dodawanie = odejmowanie, więc XOR wystarcza do obu operacji).

Dlatego algorytm LFSR (przesuń w lewo + warunkowo XOR z generatorem) to krok po kroku implementacja dzielenia wielomianów w GF(2).

B.2 Rejestr LFSR — jak algorytm z zadania działa w praktyce

LFSR (*Linear Feedback Shift Register* — liniowy rejestr przesuwający ze sprzężeniem zwrotnym) można wyobrazić sobie jako taśmę produkcyjną z 15 stanowiskami. W każdym takcie algorytmu:

1. Wszystko przesuwamy o jedno miejsce w lewo — bit ze stanowiska 0 trafia na 1, z 1 na 2, ..., z 13 na 14. Bit ze stanowiska 14 *wylatuje* z taśmy.
2. Na zwolnione stanowisko 0 wstawiamy 0.
3. Sprawdzamy, czy wylatujący bit 14 jest *różny* (XOR) od bitu wejściowego NXTBIT. Jeśli tak — wykonujemy XOR całego rejestru z 0x4599 (to jest właśnie „sprzężenie zwrotne” z nazwy LFSR).

Po przetworzeniu wszystkich 118 bitów wejścia zawartość rejestru to gotowa suma kontrolna CRC-15.

Kluczowe szczegóły w C++:

- Rejestr to zmienna `uint16_t` (16-bitowa liczba bez znaku). Używamy tylko 15 dolnych bitów — po każdym przesunięciu maskujemy: `rejestr &= 0x7FFF` (zeruje bit 15, gdyby się pojawił po przesunięciu).
- Wejście wczytujemy jako znaki `'0'/'1'`. Szybka konwersja: `znak & 1` daje wartość bitu — znaki `'0'` (kod ASCII 48) i `'1'` (kod ASCII 49) różnią się tylko ostatnim bitem.

B.3 Strategie optymalizacji algorytmu

B.3.1 Eliminacja instrukcji warunkowej (kod bez skoków)

Wewnętrzna pętla algorytmu zawiera `if`: gdy `CRCNXT = 1`, wykonaj XOR z wielomianem; gdy 0, pomiń. Nowoczesne procesory działają potokowo — zaczynają wykonywać następną instrukcję, zanim skończyły poprzednią. Gdy napotykają `if`, muszą *zgadnąć* który wariant będzie potrzebny i zacząć go wykonywać z wyprzedzeniem. Przy losowych danych wejściowych zgadują 50/50 — połowa zgadnięć jest błędna. Każdy błąd kosztuje 5–15 cykli procesora na „cofnięcie” pracy i ponowne wykonanie właściwej gałęzi. Zjawisko to nosi nazwę *branch misprediction* (błędne przewidywanie gałęzi).

Rozwiązanie: zastąpić `if` operacją arytmetyczną dającą ten sam wynik bez żadnego skoku:

- **Mnożenie:** `CRCNXT * 0x4599` — daje `0x4599` gdy `CRCNXT = 1` i `0` gdy `CRCNXT = 0`.
- **Maska bitowa:** negacja `-CRCNXT` — daje `0xFFFF` gdy `CRCNXT = 1` i `0x0000` gdy `= 0`; XOR z tą maską i wielomianem zastępuje `if`.

Przy flagie `-O3` kompilator często sam wykrywa ten wzorzec i generuje kod bezskokowy.

B.3.2 Tablica przeglądowa (zamieniamy 8 kroków w 1)

Algorytm w wersji podstawowej przetwarza wejście **bit po bicie** — 96 iteracji pętli dla 96 bitów. Można go przyspieszyć 8-krotnie, przetwarzając **bajt po bajcie** — 12 iteracji dla 96 bitów.

Idea: przed pętlą główną, raz na zawsze, oblicz co stanie się z rejestrem po przetworzeniu *każdego* możliwego bajtu. Jest ich tylko $2^8 = 256$. Zapisz wyniki w tablicy 256 elementów. W pętli głównej zamiast przetwarzać 8 bitów osobno — jedno spojrzenie do tablicy daje gotowy wynik.

Tablica ma rozmiar $256 \times 2B = 512B$, co mieści się w całości w pamięci podręcznej L1 procesora (32–64 kB). Po kilku pierwszych iteracjach cała tablica jest już w L1 — każdy odczyt trwa 1–4 cykli zamiast 100+ cykli odczytu z RAM.

Dzięki słowu kluczowemu `constexpr` tablica jest wypełniana podczas *kompilacji*, a nie podczas uruchamiania programu — zero kosztu inicjalizacji w czasie uruchomienia.

B.3.3 Flagi kompilatora

Kompilacja z `-O3 -march=native` zleca kompilatorowi agresywną optymalizację:

- **Rozwijanie pętli** (*loop unrolling*) — zamiast pętli z 12 lub 96 krokami kompilator generuje osobne instrukcje dla każdej iteracji, eliminując narzut skoków powrotnych.
- **Wbudowanie funkcji** (*inlining*) — kompilator wkleja ciało funkcji CRC w miejsce wywołania, eliminując narzut wywołania.
- **Instrukcje SIMD** — na Apple Silicon (NEON) i x86-64 (AVX2) specjalne instrukcje operują na wielu danych jednocześnie.

B.4 Pomiar czasu i integralność benchmarku

B.4.1 Mierzenie czasu

Do pomiaru czasu w C++ służy `std::chrono::high_resolution_clock` z nagłówka `<chrono>`. Zegar ten ma rozdzielczość rzędu nanosekund na macOS i Linux. Wzorzec użycia: zapisz czas startowy przed pętlą, czas końcowy po pętli, odejmij i przelicz na żadaną jednostkę (`milliseconds`, `nanoseconds` itp.) przez `duration_cast`. Dla wiarygodnych wyników powtórzeń n powinno być co najmniej 10^6 — przy mniejszych wartościach zmierzony czas jest zdominowany przez szum samego pomiaru zegara, a nie przez rzeczywisty czas algorytmu.

B.4.2 Pułapka: kompilator może usunąć całą pętlę benchmarku

Przy `-O3` kompilator analizuje kod i może stwierdzić: „wywoływana jest n razy ta sama funkcja z tym samym wejściem — wynik się nie zmienia, więc obliczę go raz i zastąpię całą pętlę jednym wywołaniem”. Benchmark pokazałby wtedy czas jednej iteracji niezależnie od wartości n — zupełnie błędny pomiar.

Dwa zabezpieczenia, które razem uniemożliwiają tę optymalizację:

- **Zmienna `volatile`** — zapisanie wyniku CRC do zmiennej `volatile` jest przez standard C++ traktowane jako „widoczny efekt uboczny”: kompilator nie może usunąć kodu, który do niej prowadzi, ani wynieść obliczeń poza pętlę.
- `__attribute__((noinline))` na funkcji CRC — blokuje wbudowanie (inlining) funkcji do miejsca wywołania. Kompilator nie „widzi” wtedy wnętrza funkcji i nie może udowodnić, że wynik jest zawsze taki sam.

Oba mechanizmy stosujemy jednocześnie jako niezależne zabezpieczenia — jedno może nie wystarczyć przy bardzo agresywnym optymalizatorze.

B.5 Wyznaczanie $T_{\min}$ i $T_{\max}$ — metoda obliczeniowa (zadanie 4.3)

Parametry bazowe

Przy prędkości 1 Mb/s każdy bit trwa $T_b = 1 \mu s$. Całkowity czas ramki to liczba bitów pomnożona przez T_b .

Stała liczba bitów ramki (bez bitów stuffujących), z tabeli w A.2:

$$L_{\text{stały}} = 1 + 32 + 6 + 64 + 16 + 2 + 7 + 3 = 131 \text{ bitów}$$

Region objęty bit stuffingiem (patrz A.3):

$$N_{\text{fill}} = 1 + 32 + 6 + 64 + 15 = 118 \text{ bitów}$$

Najkrótszy czas — zero bitów stuffujących

Żaden bit stuffujący nie zostanie dodany, gdy w regionie 118 bitów nigdzie nie pojawi się 5 identycznych bitów z rzędu. Warunek ten spełniają dane naprzemienne:

- Dane (8 bajtów): `0x55 0x55 ... (0x55 = 01010101` — ciągle przeskakiwanie 0 i 1)
- Identyfikator 29-bitowy: musi mieć naprzemienne bity (postaci `01010...`), tak aby złożenie ID, pól Control i wynikowego CRC nie tworzyło serii 5 identycznych na granicach pól

$$L_{\min} = L_{\text{stały}} + 0 = \mathbf{131} \text{ bitów}$$

$$T_{\min} = 131 \cdot 1 \mu s = \mathbf{131} \mu s$$

Współczynnik efektywności — jaki procent czasu transmisji stanowią właściwe dane (64 bity), a jaki to nagłówki, sumy kontrolne i inne pola ramki:

$$\eta_{\max} = \frac{64}{131} \approx \mathbf{48,9\%} \quad (4)$$

Najdłuższy czas — maksimum bitów stuffujących (23)

Maksymalne stuffowanie pojawia się, gdy co 5 bitów w regionie wymusza wstawienie bitu — wzorzec `11111 00000 11111 00000...`. Dla 118 bitów wejścia: $\lfloor 118/5 \rfloor = 23$ bity stuffujące.

Przykład danych generujących dużo stuffowania: same zera w polu danych `0x00` $\times 8$ to 64 bity 0 z rzędu, co samo w sobie daje $\lfloor 64/5 \rfloor = 12$ bitów stuffujących. Odpowiedni dobór ID i wynikowa suma kontrolna CRC mogą łącznie dać pełne 23.

Uwaga: wartość CRC jest obliczana automatycznie z ID i danych — nie można jej wybrać dowolnie. Znalezienie kombinacji dającej dokładnie 23 bity stuffujące wymaga iteracyjnego sprawdzania kandydatów.

$$L_{\max} = L_{\text{stały}} + 23 = 131 + 23 = \mathbf{154} \text{ bitów}$$

$$T_{\max} = 154 \cdot T_b = \mathbf{154} \mu\text{s}$$

Współczynnik efektywności dla najdłuższej ramki:

$$\eta_{\min} = \frac{64}{154} \approx \mathbf{41,6\%} \quad (5)$$

Podsumowanie zadania 4.3

Scenariusz	Bity stałe	Bity stuffuj.	Bity całk.	η
Najkrótsza ramka	131	0	131	48,9%
Najdłuższa ramka	131	23	154	41,6%

Przy 1 Mb/s: $T_{\min} = 131 \mu\text{s}$, $T_{\max} = 154 \mu\text{s}$.